



Degree: ***BSc Investment and Financial Risk Management
Bayes Business School***

Dissertation Type: Applied Research Project
Dissertation Title: ***An Empirical Comparison of Historical,
Parametric and GARCH-Based Value-at-Risk
Models for a Global Equity ETF Portfolio***

Student Name Igor Zatorski

Student ID Number:

Supervisor's name: ***Dr Malvina Marchese***

Submission date: ***11 May 2026***

"By submitting this work, I certify that I have complied with the guidelines on plagiarism outlined in the Course Handbook in the production of this dissertation and that it is my own, unaided work"

An Empirical Comparison of Historical, Parametric and GARCH-Based Value-at-Risk Models for a Global Equity ETF Portfolio

Igor Zatorski

Student ID:

BSc Investment and Financial Risk Management
Bayes Business School, City, University of London

May 2026

Supervisor: Dr Malvina Marchese

Industry collaboration: S&P Global

Industry contact: Titos Matsakos

Abstract

The aim of this project is to compare the empirical performance of five one-day Value-at-Risk models. The confidence levels are set at 95% and 99%. The approaches compared are Historical VaR, Normal parametric VaR, Student-t parametric VaR, GARCH(1,1) VaR with Normal innovations and GARCH(1,1) VaR with Student-t innovations, all estimated using a 252-day rolling calibration window. The dataset uses daily adjusted closing prices downloaded from Yahoo Finance using the `yfinance` Python library. An equal-weighted portfolio of ten global equity ETFs is constructed over the sample period from 2005 to 2025. The models are evaluated using exceedance rates, coverage error, average VaR estimates and formal backtesting tests, including Kupiec's unconditional coverage test and Christoffersen's independence and conditional coverage tests. The results show that Historical VaR achieves the lowest coverage error, while GARCH-based models perform better in terms of exceedance independence. Overall, the findings suggest a trade-off between unconditional calibration and exceedance clustering.

1 Introduction

Value-at-Risk is used in this project as a simple way to express possible portfolio losses in one number. VaR is common in risk management because it is easy to understand and because it is directly connected to a chosen time horizon and confidence level [Jorion, 2006, Hull, 2021]. However, the accuracy of VaR depends strongly on the assumptions made when modelling the loss part of the return distribution.

This matters because financial returns rarely behave like a perfect Gaussian distribution. They may show heavy tails, volatility clustering and changing risk conditions over time [Cont, 2001]. Because of this, different VaR methods may give different risk estimates for the same portfolio, especially during market stress or when the confidence level is high, such as 99%.

The goal of this project is to test several statistical approaches for one-day-ahead Value-at-Risk and compare how they perform in practice. The methods used are Historical VaR, Normal parametric VaR, Student-t parametric VaR, GARCH(1,1) VaR with Normal innovations and GARCH(1,1) VaR with Student-t innovations. These methods are estimated with a 252-day rolling calibration window. They are later tested at the 95% and 99% confidence levels on a portfolio of ten equal-weighted global equity ETFs. Their performance is compared using exceedance rates, coverage error and formal backtesting procedures, including the Kupiec and Christoffersen tests.

2 Literature Review

Jorion presents VaR as a benchmark in risk management, while Hull explains practical VaR approaches, including historical simulation and model-building, or parametric, methods [Jorion, 2006, Hull, 2021]. Historical simulation estimates VaR directly from past sample returns and therefore does not require any distributional assumptions. Parametric VaR instead assumes a distribution and estimates parameters such as the mean and volatility. In the simplest case, this distribution is assumed to be Normal [Hull, 2021].

However, Cont shows that asset returns exhibit several stylised facts, such as heavy tails, gain/loss asymmetry, volatility clustering and departures from normality [Cont, 2001]. Heavy tails indicate that extreme events may happen more often than the Normal distribution predicts. Therefore, the Student-t distribution may be more suitable for modelling asset returns, as it allows for heavier tails than the Normal one. This justifies testing Student-t and GARCH-Student-t models.

However, the distribution is not the only problem. Simple parametric models often assume constant volatility, while Cont shows that volatility clustering is an important feature of financial returns [Cont, 2001]. Engle introduces the Autoregressive Condi-

tional Heteroskedasticity (ARCH) process, where volatility is treated as something that can change over time and depend on past shocks. Bollerslev later extends this idea by adding lagged conditional variance, which gives the Generalized Autoregressive Conditional Heteroskedasticity (GARCH) model [Engle, 1982, Bollerslev, 1986].

Finally, Kupiec shows the necessity of backtesting VaR models against actual realised returns. His unconditional coverage test looks at the number of exceedances and checks if they are statistically in line with the expected number [Kupiec, 1995]. However, the Kupiec test does not look at the clustering of exceedances. Christoffersen extends this framework by testing the independence of exceedances [Christoffersen, 1998]. He introduces an independence test, which checks whether exceedances are independent over time, and also combines the Kupiec test and the independence test in order to create a conditional coverage test.

3 Models and Theoretical Framework

3.1 Value-at-Risk

Value-at-Risk is a market risk measure that estimates the potential loss of a portfolio over a given time horizon and at a given confidence level [Jorion, 2006, Hull, 2021]. In this project, the focus is on one-day-ahead VaR at the 95% and 99% confidence levels. For a long portfolio, VaR can be interpreted as a left-tail return threshold. A 95% one-day VaR corresponds to the 5% lower-tail quantile of the return distribution, while a 99% VaR corresponds to the 1% lower-tail quantile. Let c denote the confidence level, where c is equal to 0.95 or 0.99. The corresponding tail probability is then $\alpha = 1 - c$. Formally, VaR can be written as:

$$P(r_t < \text{VaR}_{c,t}) = 1 - c = \alpha,$$

where r_t is the realised portfolio return. Therefore, α is equal to 0.05 for 95% VaR and 0.01 for 99% VaR. The more negative the VaR threshold, the more conservative the risk forecast.

3.2 Historical and parametric VaR models

The first model used in this project is Historical VaR. This approach looks at past portfolio returns and creates a distribution of returns directly from them. For example, the 95% Historical VaR is based on the 5th percentile of returns in the rolling calibration window, while the 99% Historical VaR is based on the 1st percentile. The main benefit of this method is that no distributional assumptions are required. This makes it very useful as a simple benchmark.

The second approach is Parametric Normal VaR. This model assumes that portfolio log returns are normally distributed within each rolling window. The mean and standard deviation are estimated from the previous 252 daily log returns. Later, the VaR threshold is calculated using the relevant Normal quantile:

$$\text{VaR}_{c,t} = \mu_t + z_\alpha \sigma_t,$$

where μ_t is the estimated mean return, σ_t is the estimated standard deviation and z_α is the lower-tail quantile of the standard Normal distribution, with $\alpha = 1 - c$. However, financial returns often display fat tails, which means that big losses happen more often than expected by the Normal distribution [Cont, 2001]. Consequently, Normal parametric VaR might underestimate tail risk, especially at high confidence levels such as 99%.

In order to address the issue of non-normality of log returns, the third model uses the Student-t distribution. This method is similar to the Gaussian distribution, but the main difference is the allowance of fat tails through the degrees of freedom parameter. If the degrees of freedom are low, the distribution assumes fatter tails, while higher degrees of freedom make the distribution closer to the Normal distribution. In this project, the Student-t VaR model estimates the degrees of freedom, location and scale parameters over the same 252-day rolling window. However, it does not directly model time-varying volatility.

3.3 GARCH-based VaR models

One of the main features of financial returns is that volatility seems to cluster in periods of market stress. A big return, either positive or negative, is often followed by a period of higher volatility instead of an immediate return to a normal volatility level. For this reason, this project uses GARCH models, where conditional variance is modelled over time [Engle, 1982, Bollerslev, 1986].

The GARCH(1,1) model can be written as:

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2,$$

where σ_t^2 is the conditional variance, ω is the long-run variance component, ϵ_{t-1}^2 captures the impact of the previous return shock and σ_{t-1}^2 captures volatility persistence. The α parameter measures the short-term response to new shocks, while the β parameter measures the persistence of volatility over time.

This project uses two GARCH-based VaR models: GARCH(1,1)-Normal VaR and GARCH(1,1)-Student-t VaR. The GARCH-Normal model captures time-varying volatility but assumes a Normal distribution. The GARCH-Student-t model combines conditional volatility forecasting with a heavier-tailed distribution.

3.4 Backtesting framework

All models generate a one-day-ahead VaR threshold, which is then compared with the actual portfolio log return. A model is considered better calibrated if its observed exceedance rate is close to the expected tail probability. However, a good VaR model should also avoid exceedance clustering, since repeated breaches during periods of increased volatility indicate that the model is not adapting adequately to changing risk conditions. For this reason, the empirical analysis uses both exceedance rates and formal backtesting tests based on hypothesis testing. These tests include Kupiec’s test for unconditional coverage and Christoffersen’s tests for independence and conditional coverage [Kupiec, 1995, Christoffersen, 1998].

4 Data and Portfolio Construction

The dataset is built from adjusted daily prices for ten exchange-traded funds. The prices are taken from Yahoo Finance with the `yfinance` Python package. The sample starts on 3 January 2005 and ends on 31 December 2025. Adjusted prices are used because they account for dividends and other corporate actions, making them more suitable for this project than raw closing prices.

The portfolio is constructed using ten US-listed ETFs that give exposure to major global equity markets: SPY for the United States, EWA for Australia, EWC for Canada, EWG for Germany, EWH for Hong Kong, EWJ for Japan, EWL for Switzerland, EWU for the United Kingdom, EWT for Taiwan and EWZ for Brazil. All these ETFs represent local equity markets, but they are traded and denominated in US dollars. Therefore, the analysis is conducted from the perspective of a USD-based investor and does not explicitly distinguish between equity risk and foreign exchange risk. Table 1 reports the ETF constituents, their market exposures, local market currencies and listing currency.

Table 1: ETF constituents of the global equity portfolio

Ticker	Market exposure	Local market currency	Listing currency
SPY	United States	USD	USD
EWA	Australia	AUD	USD
EWC	Canada	CAD	USD
EWG	Germany	EUR	USD
EWH	Hong Kong	HKD	USD
EWJ	Japan	JPY	USD
EWL	Switzerland	CHF	USD
EWU	United Kingdom	GBP	USD
EWT	Taiwan	TWD	USD
EWZ	Brazil	BRL	USD

The portfolio is constructed on an equal-weighted basis, with a 10% allocation assigned to each ETF. Daily rebalancing is assumed in order to maintain constant weights throughout the analysis. Transaction costs, bid-ask spreads and taxes are not included; however, fund-level management costs are already reflected in the adjusted prices of each ETF. The initial portfolio value was set at \$1,000,000, which grew to approximately \$4.98 million by the end of the sample. This gives an annualised compound return of around 7.95%. Figure 1 reports how the portfolio value changes over time.

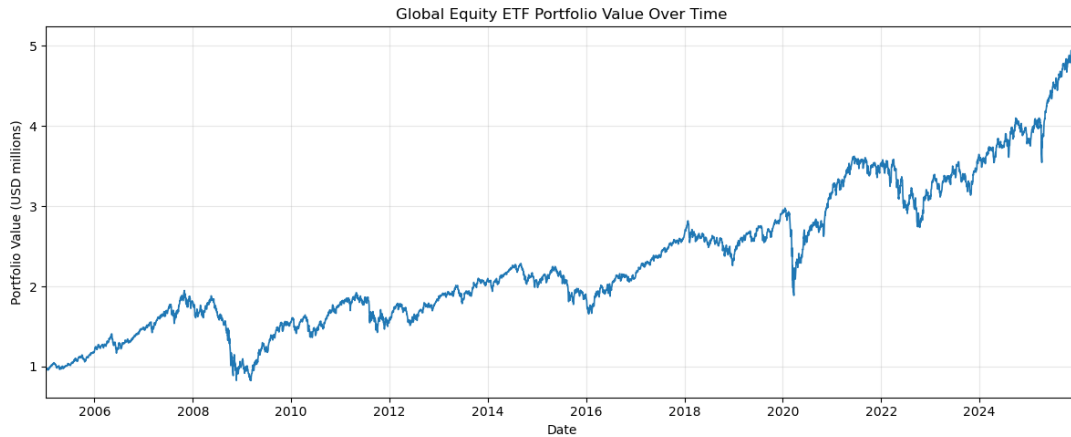


Figure 1: Value of the equal-weighted global equity ETF portfolio, 2005–2025

To update the portfolio value over time, simple arithmetic returns were used. However, for VaR estimation and formal backtesting, log returns were used, as they are more suitable for statistical risk modelling. After aligning the ETF price series and removing missing observations, the dataset contained 5,283 daily price observations. The constructed portfolio return series contained 5,281 observations, and the use of a 252-day rolling calibration window left 5,029 one-day-ahead observations for VaR backtesting. Figure 2 presents the daily portfolio log returns used for VaR estimation and backtesting.

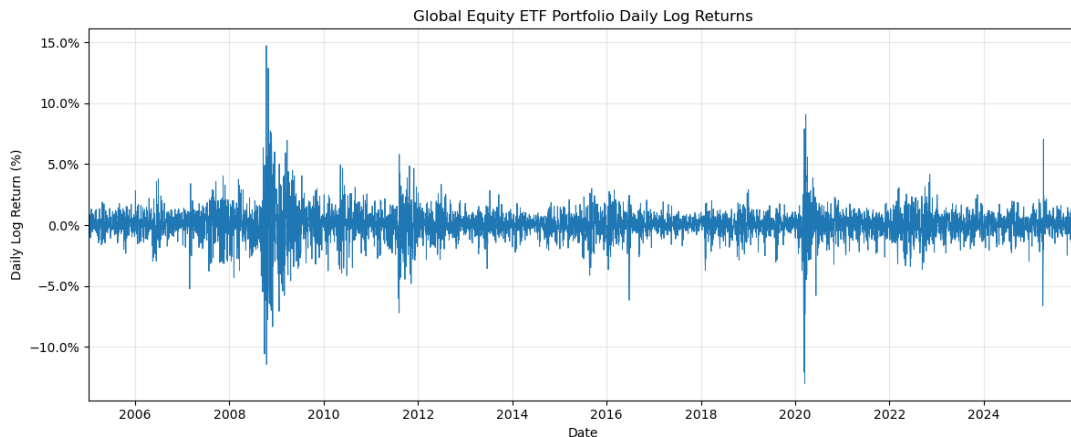


Figure 2: Daily log returns of the global equity ETF portfolio, 2005–2025

5 Empirical Methodology

The goal of the empirical analysis is to compare the accuracy of different Value-at-Risk methods used for the same portfolio consisting of global ETFs. The models are evaluated at the 95% and 99% confidence levels using one-day-ahead VaR forecasts. Five models are considered: Historical VaR, Normal parametric VaR, Student-t parametric VaR, GARCH(1,1) VaR with Normal innovations and GARCH(1,1) VaR with Student-t innovations. Each model produces a daily VaR threshold, which is then compared with the realised portfolio log return on the next trading day.

All models rely on a 252-day rolling calibration window. This window length is chosen because it approximates one trading year and allows the model parameters to adjust over time. In order to avoid look-ahead bias, each model only uses data that would have been available at that point in time. This makes the backtesting procedure closer to a real-time risk management setting. In the Historical VaR model, the VaR threshold is set as the relevant empirical quantile of past returns over the rolling window. For the Parametric Normal VaR model, the mean and standard deviation are estimated from the previous 252 daily log returns. In the Parametric Student-t VaR model, the degrees of freedom, location and scale parameters are estimated over the same window. For the GARCH models, the GARCH(1,1) specification is estimated on each rolling window. It is then used to produce the next-day forecast of the mean and volatility. This is done once with Normal innovations and once with Student-t innovations.

In this project, the VaR threshold is treated as a left-tail return threshold. An exceedance occurs when the realised portfolio log return is lower than the VaR threshold forecast by the model. Formally, the exceedance indicator is defined as:

$$I_t = \begin{cases} 1, & r_t < VaR_t, \\ 0, & r_t \geq VaR_t. \end{cases}$$

For a 95% VaR model, approximately 5% of observations are expected to breach the VaR threshold. For a 99% VaR model, the expected exceedance rate falls to approximately 1%. As a result, a well-calibrated model should produce an observed exceedance rate close to the relevant tail probability implied by the chosen confidence level.

The comparison is based on both descriptive backtesting statistics and formal statistical tests. The coverage error is calculated by looking at the absolute difference between the observed exceedance rate and the expected exceedance rate:

$$\text{Coverage Error} = |\hat{p} - p|,$$

where $\hat{p}$ is the observed exceedance rate and p is the expected tail probability. The smaller the coverage error, the better calibrated the model is in terms of unconditional ex-

ceedance frequency. However, this measure alone does not indicate whether exceedances are independent or clustered over time.

For this reason, the analysis also applies the Kupiec test for unconditional coverage and Christoffersen tests for independence and conditional coverage [Kupiec, 1995, Christoffersen, 1998]. The Kupiec test tries to determine whether the total number of VaR exceedances is statistically in line with the expected number of exceedances implied by the chosen VaR confidence level. On the other hand, Christoffersen’s test of independence looks at whether the exceedances are clustered over time during periods of increased volatility or whether they are independent. The Christoffersen conditional coverage test combines these two approaches and gives a final verdict. All statistical tests are evaluated at the 5% significance level.

6 Results and Discussion

6.1 Portfolio return behaviour

The portfolio value grew from an initial \$1 million to approximately \$4.98 million by the end of the sample. However, this growth was not smooth. The portfolio also had periods where volatility increased and the value fell strongly. The clearest examples are around the 2008 financial crisis and the COVID-19 shock in 2020. The daily log return chart also shows that the magnitude of returns changed over time. The largest moves, both positive and negative, were concentrated around periods of market stress. This suggests the presence of volatility clustering and indicates that the return distribution is unlikely to be well described by a simple constant-volatility Normal model. These features provide motivation for testing models with different distributional assumptions, such as the Student-t distribution, which allows for heavier tails, and GARCH models, which allow volatility to vary over time.

6.2 95% VaR backtesting results

Table 2 gives the results for the one-day-ahead 95% VaR forecasts. At this level, the expected breach rate is 5%, which gives 251.45 expected exceedances in the backtesting sample. Historical VaR has the lowest coverage error, with 293 exceedances and an observed exceedance rate of 5.83%. Normal VaR is close to this, with 297 exceedances and an exceedance rate of 5.91%. The other three methods have higher exceedance rates: 6.70% for GARCH(1,1)-Normal VaR, 6.88% for Student-t VaR and 6.96% for GARCH(1,1)-Student-t VaR.

Table 2: 95% VaR backtesting results

Model	Exceed.	Exp. exceed.	Exceed. rate	Coverage error	Avg. VaR	Kupiec p-value	Indep. p-value
Historical VaR	293	251.45	5.83%	0.83%	1.97%	0.0087	< 0.001
Normal VaR	297	251.45	5.91%	0.91%	1.95%	0.0041	< 0.001
GARCH(1,1)-Normal VaR	337	251.45	6.70%	1.70%	1.79%	< 0.001	0.2370
Student-t VaR	346	251.45	6.88%	1.88%	1.78%	< 0.001	< 0.001
GARCH(1,1)-Student-t VaR	350	251.45	6.96%	1.96%	1.75%	< 0.001	0.2339

Historical VaR is numerically closest to the expected exceedance rate. However, the Kupiec test still rejects it at the 5% significance level. The same result is obtained for the other models, which means that none of them gives statistically accurate unconditional coverage. Since all observed exceedance rates are above 5%, all models underestimate downside risk.

The Christoffersen independence test gives a more favourable result for the GARCH-based models. Their exceedances are not rejected as dependent over time, while the Historical VaR, Normal VaR and Student-t VaR models do reject the independence null. This suggests that GARCH helps with exceedance clustering, even though it does not solve the coverage problem.

Because all models fail the Kupiec test, all models also reject the Christoffersen conditional coverage test.

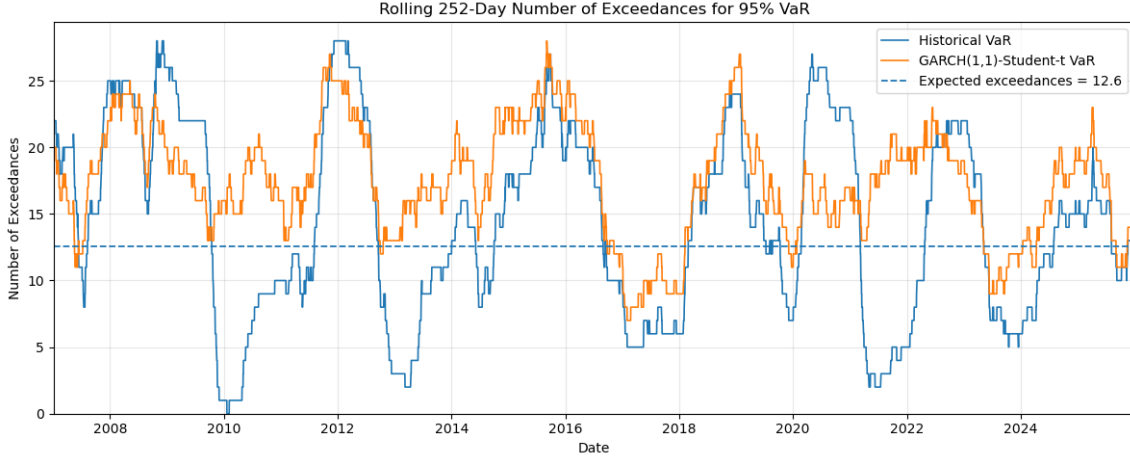


Figure 3: Rolling 252-day exceedance count for Historical VaR and GARCH(1,1)-Student-t VaR at the 95% level

6.3 99% VaR backtesting results

Table 3 presents the corresponding results for the 99% VaR models. At this level, the expected exceedance rate is 1%, corresponding to 50.29 expected exceedances. The results show that modelling extreme tail risk is more difficult. Every model produces substantially more exceedances than expected. Historical VaR again has the lowest coverage error, with 86 exceedances and an observed exceedance rate of 1.71%. Student-t

VaR and GARCH(1,1)-Student-t VaR perform next, with exceedance rates of 1.87% and 1.91%. The Normal VaR model performs worst, with 136 exceedances and an exceedance rate of 2.70%.

Table 3: 99% VaR backtesting results

Model	Exceed.	Exp. exceed.	Exceed. rate	Coverage error	Avg. VaR	Kupiec p-value	Indep. p-value
Historical VaR	86	50.29	1.71%	0.71%	3.22%	< 0.001	0.0039
Student-t VaR	94	50.29	1.87%	0.87%	3.24%	< 0.001	0.0403
GARCH(1,1)-Student-t VaR	96	50.29	1.91%	0.91%	2.76%	< 0.001	0.4196
GARCH(1,1)-Normal VaR	120	50.29	2.39%	1.39%	2.55%	< 0.001	0.9349
Normal VaR	136	50.29	2.70%	1.70%	2.76%	< 0.001	0.0013

The 99% results provide stronger evidence that the Normal distribution is not well suited to extreme tail-risk estimation. The Normal VaR model has the largest coverage error and more than twice the expected exceedance rate. The Student-t models perform better than their Normal counterparts, which is consistent with the idea that fat-tailed distributions are more appropriate for modelling extreme negative returns. Nevertheless, the Kupiec test rejects all models, so none of them achieves accurate unconditional coverage at the 99% confidence level. The independence results again favour the GARCH-based models. Both GARCH(1,1)-Normal and GARCH(1,1)-Student-t VaR fail to reject the null hypothesis of independent exceedances, while the other models reject it. As with the 95% results, all conditional coverage tests are rejected because the models fail to achieve satisfactory unconditional coverage.

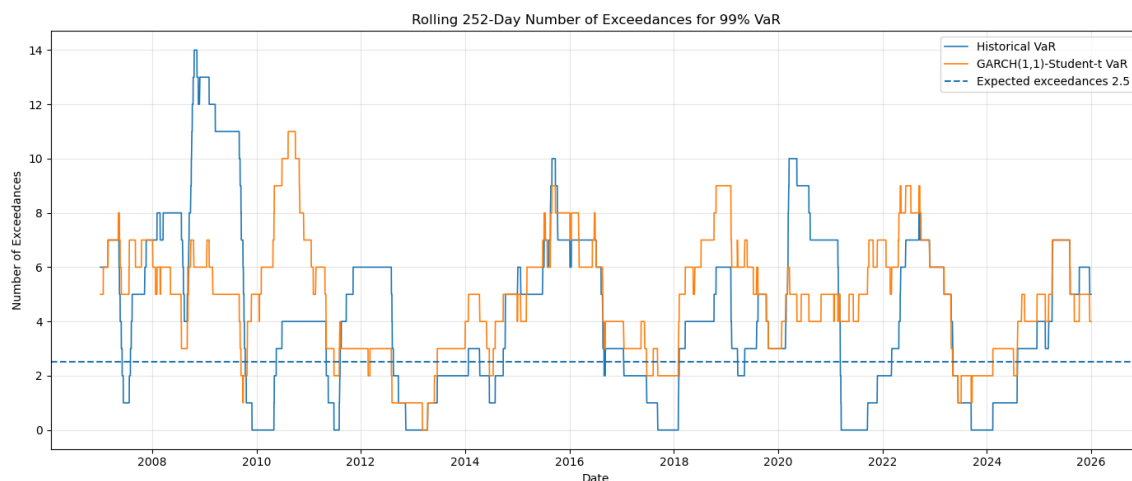


Figure 4: Rolling 252-day number of exceedances for Historical VaR and GARCH(1,1)-Student-t VaR at the 99% confidence level

6.4 Overall model comparison and discussion

Figures 3 and 4 further show that exceedances are not evenly distributed over time. Additional plots comparing realised log returns, VaR thresholds and exceedances are reported in Appendix A. The rolling number of breaches often rises above the expected level during periods of market stress, which supports the importance of testing not only unconditional coverage but also exceedance independence. Overall, the results do not identify a single model that dominates across all criteria. Historical VaR performs best in terms of coverage error at both the 95% and 99% confidence levels. This suggests that a simple empirical quantile approach can be competitive, especially when evaluated only by the closeness of observed and expected exceedance rates. However, Historical VaR fails the independence test, indicating that its exceedances are clustered during volatile periods.

The GARCH-based models show the opposite pattern. They do not achieve the best coverage accuracy, but they perform better in the Christoffersen independence test. This reflects the advantage of modelling conditional volatility: GARCH models update risk estimates more quickly after periods of market stress, reducing the clustering of VaR breaches. However, because their exceedance rates remain too high, they still underestimate downside risk overall. Therefore, the main trade-off is between unconditional calibration and exceedance independence.

The results also suggest possible extensions. First, the analysis uses only a 252-day rolling calibration window, while a longer window such as 504 trading days could be tested to examine the trade-off between responsiveness and parameter stability. Second, the GARCH specification is limited to symmetric GARCH(1,1), while asymmetric models such as EGARCH or GJR-GARCH could capture the leverage effect often observed in equity markets [Nelson, 1991, Glosten et al., 1993]. Third, more flexible tail models, such as skewed Student-t distributions, could be used to model left-tail losses more directly [Hansen, 1994]. Extreme Value Theory methods, especially approaches based on the Generalised Pareto Distribution, could also be considered as a further extension for modelling extreme tail losses [McNeil and Frey, 2000].

Another limitation of Value-at-Risk is that it only captures a loss threshold for a given confidence level. This means that it does not say how large losses can be once this threshold is exceeded. Therefore, a natural extension would be to consider Expected Shortfall (ES). This approach tries to measure the average loss when the loss is worse than the VaR threshold. As with VaR, Expected Shortfall can be estimated under different assumptions about the return distribution and time-varying volatility [Hull, 2021].

7 Conclusion

This project tested five different Value-at-Risk methods for the same one-day forecasting problem. The analysis used the 95% and 99% confidence levels and a 252-day rolling calibration window. The methods were Historical VaR, Parametric Normal VaR, Parametric Student-t VaR, GARCH(1,1)-Normal VaR and GARCH(1,1)-Student-t VaR.

Historical VaR achieved the lowest coverage error at both the 95% and 99% confidence levels. This suggests that a simple empirical quantile approach based on past returns can be competitive, even when financial returns exhibit fat tails and volatility clustering. However, this does not mean that Historical VaR is clearly the best model overall. Although it produced the closest exceedance frequency, it failed the Christoffersen independence test, indicating that its exceedances were clustered during volatile periods.

All models were rejected by the Kupiec unconditional coverage test, implying that none of them achieved statistically accurate unconditional coverage. In all cases, the models underestimated downside risk because observed exceedance rates were higher than expected. However, the GARCH-based models performed better in the Christoffersen independence test, suggesting that modelling time-varying volatility helps reduce exceedance clustering.

Overall, there is no clear winning model. Historical VaR performs best in terms of coverage error, while GARCH-based models perform better in terms of exceedance independence.

References

- Tim Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3):307–327, 1986.
- Peter F. Christoffersen. Evaluating interval forecasts. *International Economic Review*, 39(4):841–862, 1998.
- Rama Cont. Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.
- Robert F. Engle. Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4):987–1007, 1982.
- Lawrence R. Glosten, Ravi Jagannathan, and David E. Runkle. On the relation between the expected value and the volatility of the nominal excess return on stocks. *The Journal of Finance*, 48(5):1779–1801, 1993.
- Bruce E. Hansen. Autoregressive conditional density estimation. *International Economic Review*, 35(3):705–730, 1994.
- John C. Hull. *Options, Futures, and Other Derivatives*. Pearson Education Limited, 11 edition, 2021.
- Philippe Jorion. *Value at Risk: The New Benchmark for Managing Financial Risk*. McGraw-Hill, 3 edition, 2006.
- Paul H. Kupiec. Techniques for verifying the accuracy of risk measurement models. *The Journal of Derivatives*, 3(2):73–84, 1995.
- Alexander J. McNeil and Rüdiger Frey. Estimation of tail-related risk measures for heteroscedastic financial time series: An extreme value approach. *Journal of Empirical Finance*, 7(3–4):271–300, 2000.
- Daniel B. Nelson. Conditional heteroskedasticity in asset returns: A new approach. *Econometrica*, 59(2):347–370, 1991.

A Additional VaR Diagnostic Plots

This appendix presents additional diagnostic plots showing realised daily log returns, one-day-ahead VaR thresholds and exceedances for the Historical VaR and GARCH(1,1)-Student-t VaR models at the 95% and 99% confidence levels.

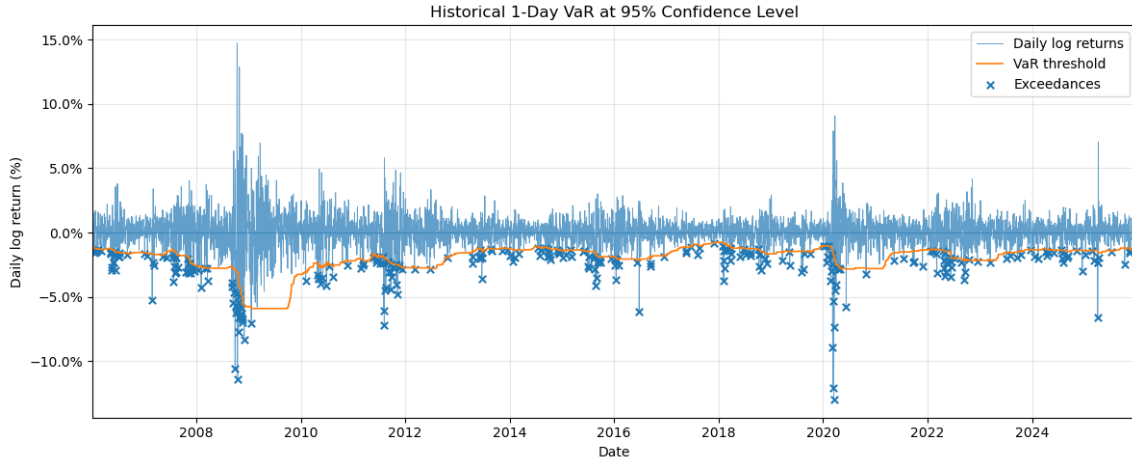


Figure 5: Historical VaR threshold and exceedances at the 95% confidence level

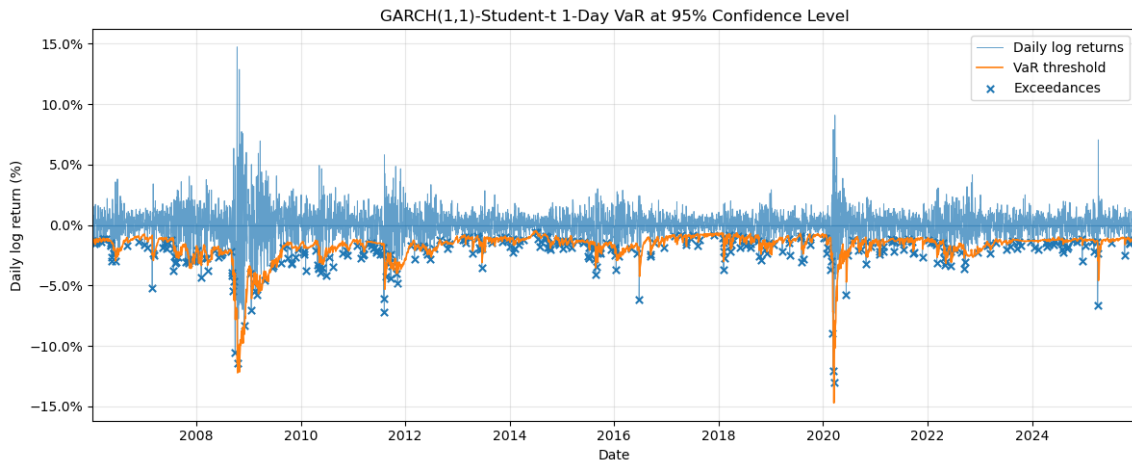


Figure 6: GARCH(1,1)-Student-t VaR threshold and exceedances at the 95% confidence level

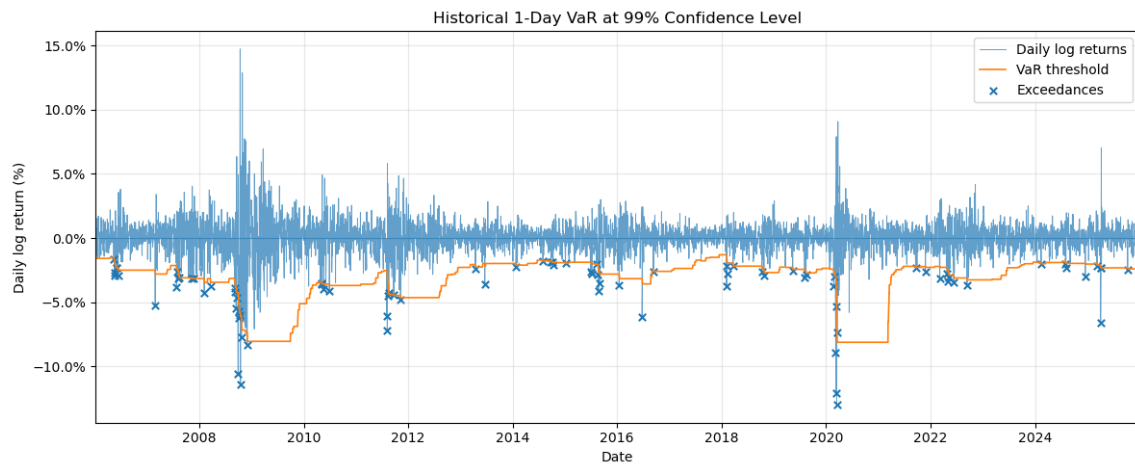


Figure 7: Historical VaR threshold and exceedances at the 99% confidence level

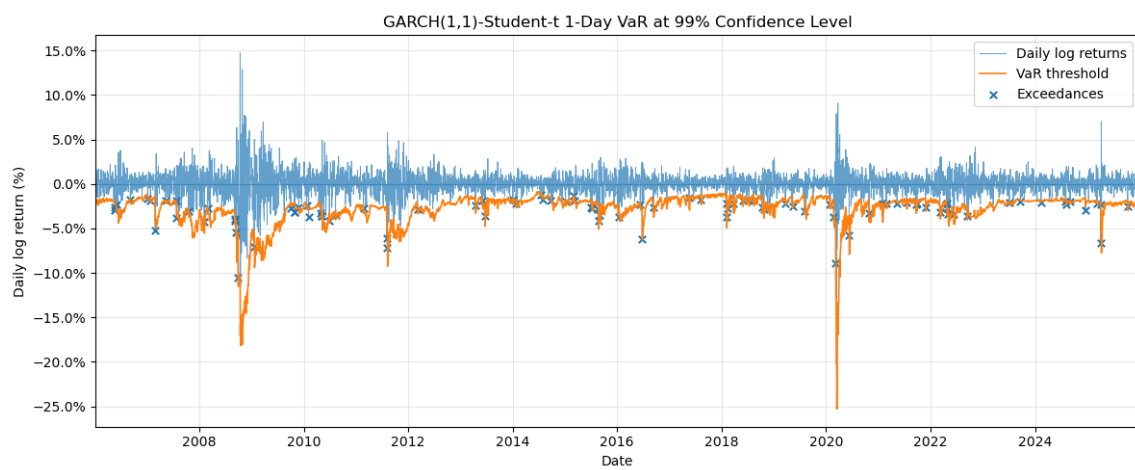


Figure 8: GARCH(1,1)-Student-t VaR threshold and exceedances at the 99% confidence level

B Python Code

The following appendix reports the Python code used for data downloading, portfolio construction, VaR estimation, backtesting and figure generation.

```
1  #1: Importing all required libraries
2  import numpy as np
3  import pandas as pd
4  import yfinance as yf
5  import matplotlib.pyplot as plt
6  from matplotlib.ticker import PercentFormatter
7  from scipy.stats import norm, t, chi2
8  from arch.univariate import arch_model
9  import os
10
11 #2: Setting global parameters
12 tickers = [
13     "SPY", # United States
14     "EWA", # Australia
15     "EWC", # Canada
16     "EWG", # Germany
17     "EWH", # Hong Kong
18     "EWJ", # Japan
19     "EWL", # Switzerland
20     "EWU", # United Kingdom
21     "EWT", # Taiwan
22     "EWZ"  # Brazil
23 ]
24
25 start_date = "2005-01-01"
26 end_date = "2026-01-01"
27 initial_value = 1_000_000
28 window = 252
29 confidence_levels = [0.95, 0.99]
30
31 #3: Creating a DataFrame describing the ETFs, their market exposures,
32 local market currencies, and
33 listing currencies
34
35 etf_info = pd.DataFrame({
36     "ticker": ["SPY", "EWA", "EWC", "EWG", "EWH", "EWJ", "EWL", "EWU",
37               "EWT", "EWZ"],
38     "market_exposure": [
39         "United States", "Australia", "Canada", "Germany", "Hong Kong",
40         "Japan", "Switzerland", "United Kingdom", "Taiwan", "Brazil"
41     ],
42     "local_market_currency": [
43         "USD", "AUD", "CAD", "EUR", "HKD",
44         "JPY", "CHF", "GBP", "TWD", "BRL"
45     ],
46     "listing_currency": ["USD"] * 10
47 })
48
49 etf_info
50
51 #4: Loading adjusted ETF prices from Yahoo Finance
```

```

49
50 def load_prices(tickers, start_date, end_date):
51     raw_data = yf.download(
52         tickers,
53         start=start_date,
54         end=end_date,
55         auto_adjust=True,
56         progress=False
57     )
58
59     clean_prices = raw_data["Close"].dropna()
60
61     return clean_prices
62
63 #5: Testing price download
64
65 prices = load_prices(tickers, start_date, end_date)
66
67 print(f"Price data shape: {prices.shape}")
68 print(f"First date: {prices.index.min().date()}")
69 print(f>Last date: {prices.index.max().date()}")
70
71 prices.head()
72
73 #6: Building an equal-weighted daily rebalanced portfolio
74
75 def build_equal_weighted_portfolio(prices, initial_value):
76     # Calculate daily simple returns for each ETF
77     individual_returns = prices.pct_change().dropna()
78
79     # Create equal weights for all ETFs
80     n_assets = individual_returns.shape[1]
81     weights = pd.Series(1 / n_assets, index=individual_returns.columns)
82
83     # Daily rebalanced portfolio return
84     portfolio_simple_return = individual_returns.dot(weights)
85
86     # Convert daily portfolio returns into portfolio value
87     portfolio_value = initial_value * (1 + portfolio_simple_return).
88         cumprod()
89
90     # Store portfolio value and returns in one DataFrame
91     portfolio = pd.DataFrame({
92         "price": portfolio_value,
93         "simple_ret": portfolio_simple_return
94     })
95
96     # Calculate log returns for VaR modelling
97     portfolio["log_ret"] = np.log(portfolio["price"] / portfolio["price
98         "].shift(1))
99
100     # Remove the first missing log return
101     portfolio = portfolio.dropna()
102
103     return portfolio
104
105 #7: Testing portfolio construction

```

```

105 portfolio = build_equal_weighted_portfolio(prices, initial_value)
106
107 print(f"Portfolio data shape: {portfolio.shape}")
108 print(f"First portfolio date: {portfolio.index.min().date()}")
109 print(f"Last portfolio date: {portfolio.index.max().date()}")
110 print(f"Initial capital: ${initial_value:,.2f}")
111 print(f"Final portfolio value: ${portfolio['price'].iloc[-1]:,.2f}")
112
113 portfolio.head()
114
115 #8: Creating a standard output format for VaR models
116
117 def make_var_output(portfolio, var_threshold, model_name,
118                    confidence_level, window):
119     result = pd.DataFrame(index=portfolio.index)
120
121     result["price"] = portfolio["price"]
122     result["log_ret"] = portfolio["log_ret"]
123     result["var_threshold"] = var_threshold
124
125     result = result.dropna(subset=["var_threshold"]).copy()
126
127     result["var_positive"] = -result["var_threshold"]
128     result["exceedance"] = result["log_ret"] < result["var_threshold"]
129
130     result["model"] = model_name
131     result["confidence_level"] = confidence_level
132     result["tail_prob"] = 1 - confidence_level
133     result["window"] = window
134
135     return result
136
137 #9: Historical 1-day VaR with rolling calibration window
138
139 def historical_var(portfolio, window, confidence_level):
140     # Convert confidence level into tail probability
141     tail_prob = 1 - confidence_level
142
143     # Calculate historical VaR threshold using past rolling returns
144     # only
145     var_threshold = (
146         portfolio["log_ret"]
147         .shift(1)
148         .rolling(window=window)
149         .quantile(tail_prob)
150     )
151
152     # Return results in the standard VaR output format
153     return make_var_output(
154         portfolio=portfolio,
155         var_threshold=var_threshold,
156         model_name="Historical VaR",
157         confidence_level=confidence_level,
158         window=window
159     )
160
161 #10: Testing Historical VaR

```

```

161 hist_95 = historical_var(portfolio, window, 0.95)
162 hist_99 = historical_var(portfolio, window, 0.99)
163
164 print(f"Historical VaR 95% observations: {len(hist_95)}")
165 print(f"Historical VaR 99% observations: {len(hist_99)}")
166
167 hist_95.head()
168
169 #11: Parametric Normal 1-day VaR with rolling calibration window
170
171 def normal_var(portfolio, window, confidence_level):
172     # Convert confidence level into tail probability
173     tail_prob = 1 - confidence_level
174
175     # Calculate rolling mean using past returns only
176     rolling_mean = (
177         portfolio["log_ret"]
178         .shift(1)
179         .rolling(window=window)
180         .mean()
181     )
182
183     # Calculate rolling standard deviation using past returns only
184     rolling_std = (
185         portfolio["log_ret"]
186         .shift(1)
187         .rolling(window=window)
188         .std()
189     )
190
191     # Normal left-tail quantile
192     z_score = norm.ppf(tail_prob)
193
194     # Parametric Normal VaR threshold
195     var_threshold = rolling_mean + rolling_std * z_score
196
197     # Return results in the standard VaR output format
198     return make_var_output(
199         portfolio=portfolio,
200         var_threshold=var_threshold,
201         model_name="Normal VaR",
202         confidence_level=confidence_level,
203         window=window
204     )
205
206 #12: Testing Normal VaR
207
208 normal_95 = normal_var(portfolio, window, 0.95)
209 normal_99 = normal_var(portfolio, window, 0.99)
210
211 print(f"Normal VaR 95% observations: {len(normal_95)}")
212 print(f"Normal VaR 99% observations: {len(normal_99)}")
213
214 normal_95.head()
215
216 #13: Parametric Student-t 1-day VaR with rolling calibration window
217
218 def student_t_var(portfolio, window, confidence_level):

```

```

219     # Convert confidence level into tail probability
220     tail_prob = 1 - confidence_level
221
222     # Prepare empty Series for VaR threshold
223     var_threshold = pd.Series(index=portfolio.index, dtype=float)
224
225     # Rolling estimation of Student-t parameters
226     for i in range(window, len(portfolio)):
227         # Use only past returns to avoid look-ahead bias
228         sample = portfolio["log_ret"].iloc[i-window:i]
229
230         # Fit Student-t distribution to the rolling window
231         nu, loc, scale = t.fit(sample)
232
233         # Calculate left-tail quantile of fitted Student-t distribution
234         var_threshold.iloc[i] = t.ppf(
235             tail_prob,
236             df=nu,
237             loc=loc,
238             scale=scale
239         )
240
241     # Return results in the standard VaR output format
242     return make_var_output(
243         portfolio=portfolio,
244         var_threshold=var_threshold,
245         model_name="Student-t VaR",
246         confidence_level=confidence_level,
247         window=window
248     )
249
250 #14: Testing Student-t VaR
251
252 student_95 = student_t_var(portfolio, window, 0.95)
253 student_99 = student_t_var(portfolio, window, 0.99)
254
255 print(f"Student-t VaR 95% observations: {len(student_95)}")
256 print(f"Student-t VaR 99% observations: {len(student_99)}")
257
258 student_95.head()
259
260 #15: GARCH(1,1)-Normal 1-day VaR with rolling calibration window
261
262 def garch_normal_var(portfolio, window, confidence_level):
263     # Convert confidence level into tail probability
264     tail_prob = 1 - confidence_level
265
266     # Prepare empty Series for VaR threshold
267     var_threshold = pd.Series(index=portfolio.index, dtype=float)
268
269     # Rolling GARCH estimation
270     for i in range(window, len(portfolio)):
271         # Use only past returns to avoid look-ahead bias
272         # Multiply by 100 because the arch package works better with
273             percentage returns
274         sample = portfolio["log_ret"].iloc[i-window:i] * 100
275
276         # Define GARCH(1,1) model with Normal innovations

```

```

276     model = arch_model(
277         sample,
278         mean="Constant",
279         vol="GARCH",
280         p=1,
281         q=1,
282         dist="normal",
283         rescale=False
284     )
285
286     # Fit the model
287     fitted_model = model.fit(
288         disp="off",
289         update_freq=0,
290         show_warning=False
291     )
292
293     # Forecast one-day-ahead conditional mean and variance
294     forecast = fitted_model.forecast(horizon=1, reindex=False)
295
296     mu_forecast = forecast.mean.iloc[-1, 0]
297     sigma_forecast = np.sqrt(forecast.variance.iloc[-1, 0])
298
299     # Normal left-tail quantile
300     q = norm.ppf(tail_prob)
301
302     # GARCH-Normal VaR threshold, converted back from percentage to
303     # decimal returns
304     var_threshold.iloc[i] = (mu_forecast + sigma_forecast * q) /
305                             100
306
307     # Return results in the standard VaR output format
308     return make_var_output(
309         portfolio=portfolio,
310         var_threshold=var_threshold,
311         model_name="GARCH(1,1)-Normal VaR",
312         confidence_level=confidence_level,
313         window=window
314     )
315
316 #16: Testing GARCH(1,1)-Normal VaR
317
318 garch_normal_95 = garch_normal_var(portfolio, window, 0.95)
319 garch_normal_99 = garch_normal_var(portfolio, window, 0.99)
320
321 print(f"GARCH-Normal VaR 95% observations: {len(garch_normal_95)}")
322 print(f"GARCH-Normal VaR 99% observations: {len(garch_normal_99)}")
323
324 garch_normal_95.head()
325
326 #17: GARCH(1,1)-Student-t 1-day VaR with rolling calibration window
327
328 def garch_t_var(portfolio, window, confidence_level):
329     # Convert confidence level into tail probability
330     tail_prob = 1 - confidence_level
331
332     # Prepare empty Series for VaR threshold

```

```

332     var_threshold = pd.Series(index=portfolio.index, dtype=float)
333
334     # Rolling GARCH estimation
335     for i in range(window, len(portfolio)):
336         # Use only past returns to avoid look-ahead bias
337         # Multiply by 100 because arch works better with percentage
338             returns
339         sample = portfolio["log_ret"].iloc[i-window:i] * 100
340
341         # Define GARCH(1,1) model with Student-t innovations
342         model = arch_model(
343             sample,
344             mean="Constant",
345             vol="GARCH",
346             p=1,
347             q=1,
348             dist="t",
349             rescale=False
350         )
351
352         # Fit the model
353         fitted_model = model.fit(
354             disp="off",
355             update_freq=0,
356             show_warning=False
357         )
358
359         # Forecast one-day-ahead conditional mean and variance
360         forecast = fitted_model.forecast(horizon=1, reindex=False)
361
362         mu_forecast = forecast.mean.iloc[-1, 0]
363         sigma_forecast = np.sqrt(forecast.variance.iloc[-1, 0])
364
365         # Extract Student-t degrees of freedom
366         nu = fitted_model.params["nu"]
367
368         # Student-t left-tail quantile
369         # The arch package uses standardised Student-t innovations,
370         # so the scipy t quantile is rescaled to have variance equal to
371             1.
372         q = t.ppf(tail_prob, df=nu) * np.sqrt((nu - 2) / nu)
373
374         # GARCH-t VaR threshold, converted back from percentage to
375             decimal returns
376         var_threshold.iloc[i] = (mu_forecast + sigma_forecast * q) /
377             100
378
379         # Return results in the standard VaR output format
380         return make_var_output(
381             portfolio=portfolio,
382             var_threshold=var_threshold,
383             model_name="GARCH(1,1)-Student-t VaR",
384             confidence_level=confidence_level,
385             window=window
386         )
387
388 #18: Testing GARCH(1,1)-Student-t VaR
389

```

```

386 garch_t_95 = garch_t_var(portfolio, window, 0.95)
387 garch_t_99 = garch_t_var(portfolio, window, 0.99)
388
389 print(f"GARCH-t VaR 95% observations: {len(garch_t_95)}")
390 print(f"GARCH-t VaR 99% observations: {len(garch_t_99)}")
391
392 garch_t_95.head()
393
394 #19: Summarising VaR model results
395
396 def summarise_var_results(var_result):
397     model = var_result["model"].iloc[0]
398     confidence_level = var_result["confidence_level"].iloc[0]
399     tail_prob = var_result["tail_prob"].iloc[0]
400     window = var_result["window"].iloc[0]
401
402     n_obs = len(var_result)
403     n_exceed = int(var_result["exceedance"].sum())
404     expected_exceed = n_obs * tail_prob
405     exceedance_rate = n_exceed / n_obs
406     expected_rate = tail_prob
407     empirical_coverage = 1 - exceedance_rate
408     average_var = var_result["var_positive"].mean()
409
410     summary = pd.DataFrame({
411         "model": [model],
412         "confidence_level": [confidence_level],
413         "window": [window],
414         "n_obs": [n_obs],
415         "n_exceed": [n_exceed],
416         "expected_exceed": [expected_exceed],
417         "exceedance_rate": [exceedance_rate],
418         "expected_rate": [expected_rate],
419         "empirical_coverage": [empirical_coverage],
420         "average_var": [average_var]
421     })
422
423     return summary
424
425 #20: Comparing all VaR models
426
427 all_var_results = [
428     hist_95, hist_99,
429     normal_95, normal_99,
430     student_95, student_99,
431     garch_normal_95, garch_normal_99,
432     garch_t_95, garch_t_99
433 ]
434
435 comparison = pd.concat(
436     [summarise_var_results(result) for result in all_var_results],
437     ignore_index=True
438 )
439
440 comparison_formatted = comparison.copy()
441
442 comparison_formatted["confidence_level"] = comparison_formatted["
confidence_level"].map(lambda x: f"

```



```

443         {x:.0%}")
comparison_formatted["expected_exceed"] = comparison_formatted["
        expected_exceed"].map(lambda x: f"{
444         x:.2f}")
comparison_formatted["exceedance_rate"] = comparison_formatted["
        exceedance_rate"].map(lambda x: f"{
445         x:.2%}")
comparison_formatted["expected_rate"] = comparison_formatted["
        expected_rate"].map(lambda x: f"{x
446         :.2%}")
comparison_formatted["empirical_coverage"] = comparison_formatted["
        empirical_coverage"].map(lambda x:
447         f"{x:.2%}")
comparison_formatted["average_var"] = comparison_formatted["average_var
        "].map(lambda x: f"{x:.2%}")
448
449 comparison_formatted
450
451 #21: Separate comparison tables for 95% and 99% VaR
452
453 comparison_with_error = comparison.copy()
454
455 # Absolute distance between actual and expected exceedance rate
456 comparison_with_error["coverage_error"] = (
457     comparison_with_error["exceedance_rate"]
458     - comparison_with_error["expected_rate"]
459 ).abs()
460
461 # Select clean columns
462 columns_to_show = [
463     "model",
464     "window",
465     "n_obs",
466     "n_exceed",
467     "expected_exceed",
468     "exceedance_rate",
469     "expected_rate",
470     "coverage_error",
471     "empirical_coverage",
472     "average_var"
473 ]
474
475 # Create separate raw numeric tables
476 comparison_95 = (
477     comparison_with_error[comparison_with_error["confidence_level"] ==
478         0.95]
479     [columns_to_show]
480     .sort_values("coverage_error")
481     .reset_index(drop=True)
482 )
483 comparison_99 = (
484     comparison_with_error[comparison_with_error["confidence_level"] ==
485         0.99]
486     [columns_to_show]
487     .sort_values("coverage_error")
488     .reset_index(drop=True)
489 )

```

```

489
490 # Formatting function for display only
491 def format_comparison_table(df):
492     formatted = df.copy()
493
494     formatted["expected_exceed"] = formatted["expected_exceed"].map(
495         lambda x: f"{x:.2f}")
496     formatted["exceedance_rate"] = formatted["exceedance_rate"].map(
497         lambda x: f"{x:.2%}")
498     formatted["expected_rate"] = formatted["expected_rate"].map(lambda
499         x: f"{x:.2%}")
500     formatted["coverage_error"] = formatted["coverage_error"].map(
501         lambda x: f"{x:.2%}")
502     formatted["empirical_coverage"] = formatted["empirical_coverage"].
503         map(lambda x: f"{x:.2%}")
504     formatted["average_var"] = formatted["average_var"].map(lambda x: f
505         "{x:.2%}")
506
507     return formatted
508
509 # Create formatted display tables
510 comparison_95_formatted = format_comparison_table(comparison_95)
511 comparison_99_formatted = format_comparison_table(comparison_99)
512
513 print("95% VaR Model Comparison")
514 comparison_95_formatted
515
516 print("99% VaR Model Comparison")
517 comparison_99_formatted
518
519 #22: Kupiec unconditional coverage test
520
521 def kupiec_test(var_result):
522     # Extract exceedance indicators as 0/1 values
523     exceedances = var_result["exceedance"].astype(int)
524
525     # Number of observations and number of exceedances
526     n_obs = len(exceedances)
527     n_exceed = int(exceedances.sum())
528
529     # Expected exceedance probability
530     tail_prob = var_result["tail_prob"].iloc[0]
531
532     # Observed exceedance probability
533     observed_prob = n_exceed / n_obs
534
535     # Helper function for binomial log-likelihood
536     def binomial_log_likelihood(prob, n_exceed, n_obs):
537         if prob == 0:
538             return 0 if n_exceed == 0 else -np.inf
539         if prob == 1:
540             return 0 if n_exceed == n_obs else -np.inf
541
542         return (
543             n_exceed * np.log(prob)
544             + (n_obs - n_exceed) * np.log(1 - prob)
545         )

```

```

541     # Log-likelihood under the null hypothesis: exceedance probability
542         = tail_prob
543     log_likelihood_null = binomial_log_likelihood(
544         tail_prob,
545         n_exceed,
546         n_obs
547     )
548     # Log-likelihood under the alternative hypothesis: exceedance
549         probability = observed_prob
550     log_likelihood_alt = binomial_log_likelihood(
551         observed_prob,
552         n_exceed,
553         n_obs
554     )
555     # Kupiec likelihood ratio statistic
556     lr_uc = -2 * (log_likelihood_null - log_likelihood_alt)
557
558     # p-value from chi-square distribution with 1 degree of freedom
559     p_value = 1 - chi2.cdf(lr_uc, df=1)
560
561     return pd.DataFrame({
562         "kupiec_lr": [lr_uc],
563         "kupiec_p_value": [p_value],
564         "kupiec_reject_5pct": [p_value < 0.05]
565     })
566
567 #23: Add Kupiec test results to all model summaries
568
569 def summarise_var_results_with_kupiec(var_result):
570     summary = summarise_var_results(var_result)
571     kupiec = kupiec_test(var_result)
572
573     return pd.concat(
574         [summary.reset_index(drop=True), kupiec.reset_index(drop=True)]
575         ,
576         axis=1
577     )
578
579 #24: Comparing all VaR models with Kupiec test
580
581 comparison_kupiec = pd.concat(
582     [summarise_var_results_with_kupiec(result) for result in
583         all_var_results],
584     ignore_index=True
585 )
586
587 comparison_kupiec
588
589 #25: Formatted 95% and 99% comparison tables with Kupiec test
590
591 comparison_kupiec["coverage_error"] = (
592     comparison_kupiec["exceedance_rate"]
593     - comparison_kupiec["expected_rate"]
594 ).abs()
595
596 columns_to_show = [

```

```

595     "model",
596     "n_obs",
597     "n_exceed",
598     "expected_exceed",
599     "exceedance_rate",
600     "expected_rate",
601     "coverage_error",
602     "empirical_coverage",
603     "average_var",
604     "kupiec_lr",
605     "kupiec_p_value",
606     "kupiec_reject_5pct"
607 ]
608
609 comparison_95_kupiec = (
610     comparison_kupiec[comparison_kupiec["confidence_level"] == 0.95]
611     [columns_to_show]
612     .sort_values("coverage_error")
613     .reset_index(drop=True)
614 )
615
616 comparison_99_kupiec = (
617     comparison_kupiec[comparison_kupiec["confidence_level"] == 0.99]
618     [columns_to_show]
619     .sort_values("coverage_error")
620     .reset_index(drop=True)
621 )
622
623 def format_kupiec_table(df):
624     formatted = df.copy()
625
626     formatted["expected_exceed"] = formatted["expected_exceed"].map(
627         lambda x: f"{x:.2f}")
628     formatted["exceedance_rate"] = formatted["exceedance_rate"].map(
629         lambda x: f"{x:.2%}")
630     formatted["expected_rate"] = formatted["expected_rate"].map(lambda x: f"{x:.2%}")
631     formatted["coverage_error"] = formatted["coverage_error"].map(
632         lambda x: f"{x:.2%}")
633     formatted["empirical_coverage"] = formatted["empirical_coverage"].map(
634         lambda x: f"{x:.2%}")
635     formatted["average_var"] = formatted["average_var"].map(lambda x: f"{x:.2%}")
636     formatted["kupiec_lr"] = formatted["kupiec_lr"].map(lambda x: f"{x:.4f}")
637     formatted["kupiec_p_value"] = formatted["kupiec_p_value"].map(
638         lambda x: f"{x:.4g}")
639
640     return formatted
641
642 comparison_95_kupiec_formatted = format_kupiec_table(
643     comparison_95_kupiec
644 )
645 comparison_99_kupiec_formatted = format_kupiec_table(
646     comparison_99_kupiec
647 )
648
649 print("95% VaR Model Comparison with Kupiec Test")
650 comparison_95_kupiec_formatted

```

```

643 print("99% VaR Model Comparison with Kupiec Test")
644 comparison_99_kupiec_formatted
645
646 #26: Christoffersen independence and conditional coverage tests
647
648 def christoffersen_test(var_result):
649     # Convert exceedance indicator to integer 0/1
650     exceedances = var_result["exceedance"].astype(int).values
651
652     # Previous and current exceedance indicators
653     previous = exceedances[:-1]
654     current = exceedances[1:]
655
656     # Transition counts
657     n00 = np.sum((previous == 0) & (current == 0))
658     n01 = np.sum((previous == 0) & (current == 1))
659     n10 = np.sum((previous == 1) & (current == 0))
660     n11 = np.sum((previous == 1) & (current == 1))
661
662     # Safe log-likelihood helper
663     def safe_log_likelihood(count_success, count_failure, prob):
664         if prob <= 0:
665             return 0 if count_success == 0 else -np.inf
666         if prob >= 1:
667             return 0 if count_failure == 0 else -np.inf
668
669         return (
670             count_success * np.log(prob)
671             + count_failure * np.log(1 - prob)
672         )
673
674     # Unconditional exceedance probability in transition sample
675     total_transitions = n00 + n01 + n10 + n11
676     pi = (n01 + n11) / total_transitions
677
678     # Conditional exceedance probabilities
679     pi0 = n01 / (n00 + n01) if (n00 + n01) > 0 else 0
680     pi1 = n11 / (n10 + n11) if (n10 + n11) > 0 else 0
681
682     # Log-likelihood under independence
683     log_likelihood_independent = (
684         safe_log_likelihood(n01 + n11, n00 + n10, pi)
685     )
686
687     # Log-likelihood under Markov dependence
688     log_likelihood_dependent = (
689         safe_log_likelihood(n01, n00, pi0)
690         + safe_log_likelihood(n11, n10, pi1)
691     )
692
693     # Christoffersen independence likelihood ratio statistic
694     lr_ind = -2 * (log_likelihood_independent -
695                    log_likelihood_dependent)
696
697     p_value_ind = 1 - chi2.cdf(lr_ind, df=1)
698
699     # Kupiec unconditional coverage statistic
700     kupiec = kupiec_test(var_result)
701     lr_uc = kupiec["kupiec_lr"].iloc[0]

```

```

700
701     # Conditional coverage statistic = unconditional coverage +
                                         independence
702     lr_cc = lr_uc + lr_ind
703     p_value_cc = 1 - chi2.cdf(lr_cc, df=2)
704
705     return pd.DataFrame({
706         "n00": [n00],
707         "n01": [n01],
708         "n10": [n10],
709         "n11": [n11],
710         "pi0": [pi0],
711         "pi1": [pi1],
712         "christoffersen_ind_lr": [lr_ind],
713         "christoffersen_ind_p_value": [p_value_ind],
714         "christoffersen_ind_reject_5pct": [p_value_ind < 0.05],
715         "christoffersen_cc_lr": [lr_cc],
716         "christoffersen_cc_p_value": [p_value_cc],
717         "christoffersen_cc_reject_5pct": [p_value_cc < 0.05]
718     })
719
720 #27: Clean backtesting tables with Kupiec and Christoffersen tests
721
722 def format_p_value(p):
723     if p < 0.001:
724         return "<0.001"
725     else:
726         return f"{p:.4f}"
727
728
729 def format_rejection(reject):
730     if reject:
731         return "Reject"
732     else:
733         return "Do not reject"
734
735
736 def summarise_all_backtests(var_result):
737     summary = summarise_var_results(var_result)
738     kupiec = kupiec_test(var_result)
739     christoffersen = christoffersen_test(var_result)
740
741     result = pd.concat(
742         [
743             summary.reset_index(drop=True),
744             kupiec.reset_index(drop=True),
745             christoffersen.reset_index(drop=True)
746         ],
747         axis=1
748     )
749
750     result["coverage_error"] = (
751         result["exceedance_rate"] - result["expected_rate"]
752     ).abs()
753
754     return result
755
756 #28: Run all backtests for all models

```

```

757
758 backtesting_results = pd.concat(
759     [summarise_all_backtests(result) for result in all_var_results],
760     ignore_index=True
761 )
762
763 backtesting_results
764
765 #29: Create clean formatted 95% and 99% backtesting tables
766
767 def create_clean_backtesting_table(backtesting_results,
768                                     confidence_level):
769
770     table = backtesting_results[
771         backtesting_results["confidence_level"] == confidence_level
772     ].copy()
773
774     table = table.sort_values("coverage_error").reset_index(drop=True)
775
776     columns_to_keep = [
777         "model",
778         "n_exceed",
779         "expected_exceed",
780         "exceedance_rate",
781         "coverage_error",
782         "average_var",
783         "kupiec_p_value",
784         "kupiec_reject_5pct",
785         "christoffersen_ind_p_value",
786         "christoffersen_ind_reject_5pct",
787         "christoffersen_cc_p_value",
788         "christoffersen_cc_reject_5pct"
789     ]
790
791     table = table[columns_to_keep]
792
793     table = table.rename(columns={
794         "model": "Model",
795         "n_exceed": "Exceedances",
796         "expected_exceed": "Expected exceedances",
797         "exceedance_rate": "Exceedance rate",
798         "coverage_error": "Coverage error",
799         "average_var": "Average VaR",
800         "kupiec_p_value": "Kupiec p-value",
801         "kupiec_reject_5pct": "Kupiec decision",
802         "christoffersen_ind_p_value": "Independence p-value",
803         "christoffersen_ind_reject_5pct": "Independence decision",
804         "christoffersen_cc_p_value": "Conditional coverage p-value",
805         "christoffersen_cc_reject_5pct": "Conditional coverage decision"
806     })
807
808     table["Expected exceedances"] = table["Expected exceedances"].map(
809         lambda x: f"{x:.2f}")
810
811     table["Exceedance rate"] = table["Exceedance rate"].map(lambda x: f"{x:.2%}")
812
813     table["Coverage error"] = table["Coverage error"].map(lambda x: f"{x:.2%}")
814
815     table["Average VaR"] = table["Average VaR"].map(lambda x: f"{x:.2%}")

```

```

810         ")
811     table["Kupiec p-value"] = table["Kupiec p-value"].map(
812         format_p_value)
813     table["Independence p-value"] = table["Independence p-value"].map(
814         format_p_value)
815     table["Conditional coverage p-value"] = table["Conditional coverage
816         p-value"].map(format_p_value)
817     table["Kupiec decision"] = table["Kupiec decision"].map(
818         format_rejection)
819     table["Independence decision"] = table["Independence decision"].map(
820         format_rejection)
821     table["Conditional coverage decision"] = table["Conditional
822         coverage decision"].map(
823         format_rejection)
824
825     return table
826
827 backtesting_95_formatted = create_clean_backtesting_table(
828     backtesting_results, 0.95)
829 backtesting_99_formatted = create_clean_backtesting_table(
830     backtesting_results, 0.99)
831
832 print("95% VaR Backtesting Results")
833 backtesting_95_formatted
834
835 print("99% VaR Backtesting Results")
836 backtesting_99_formatted
837
838 #30: Plot portfolio value over time
839
840 figure_dir = "figures"
841 os.makedirs(figure_dir, exist_ok=True)
842
843 fig, ax = plt.subplots(figsize=(12, 5))
844
845 ax.plot(
846     portfolio.index,
847     portfolio["price"] / 1_000_000,
848     linewidth=1.2
849 )
850
851 ax.set_title("Global Equity ETF Portfolio Value Over Time")
852 ax.set_xlabel("Date")
853 ax.set_ylabel("Portfolio Value (USD millions)")
854
855 ax.grid(True, alpha=0.3)
856 ax.margins(x=0)
857
858 plt.tight_layout()
859 plt.savefig(f"{figure_dir}/portfolio_value_over_time.png", dpi=300,
860             bbox_inches="tight")
861
862 plt.show()
863
864 #31: Plot daily portfolio log returns

```



```

857 figure_dir = "figures"
858 os.makedirs(figure_dir, exist_ok=True)
859
860 fig, ax = plt.subplots(figsize=(12, 5))
861
862 ax.plot(
863     portfolio.index,
864     portfolio["log_ret"],
865     linewidth=0.7
866 )
867
868 ax.axhline(
869     0,
870     linestyle="--",
871     linewidth=0.8,
872     alpha=0.7
873 )
874
875 ax.set_title("Global Equity ETF Portfolio Daily Log Returns")
876 ax.set_xlabel("Date")
877 ax.set_ylabel("Daily Log Return (%)")
878
879 ax.yaxis.set_major_formatter(PercentFormatter(1.0))
880
881 ax.grid(True, alpha=0.3)
882 ax.margins(x=0)
883
884 plt.tight_layout()
885 plt.savefig(f"{figure_dir}/portfolio_log_returns.png", dpi=300,
886             bbox_inches="tight")
887
888 #32: Function for plotting VaR thresholds and exceedances
889
890 def plot_var_model(var_result, title, filename, y_limits=None):
891     data = var_result.dropna(subset=["log_ret", "var_threshold", "
892                                 exceedance"]).copy()
893
894     data["exceedance"] = data["exceedance"].astype(bool)
895
896     exceedances = data[data["exceedance"]]
897
898     fig, ax = plt.subplots(figsize=(12, 5))
899
900     ax.plot(
901         data.index,
902         data["log_ret"],
903         linewidth=0.7,
904         alpha=0.7,
905         label="Daily log returns"
906     )
907
908     ax.plot(
909         data.index,
910         data["var_threshold"],
911         linewidth=1.2,
912         label="VaR threshold"
913     )

```

```

913     ax.scatter(
914         exceedances.index,
915         exceedances["log_ret"],
916         marker="x",
917         s=28,
918         label="Exceedances"
919     )
920
921     ax.axhline(
922         0,
923         linewidth=0.8,
924         alpha=0.5
925     )
926
927     ax.set_title(title)
928     ax.set_xlabel("Date")
929     ax.set_ylabel("Daily log return (%)")
930     ax.yaxis.set_major_formatter(PercentFormatter(1.0))
931
932     if y_limits is not None:
933         ax.set_ylim(y_limits)
934
935     ax.margins(x=0)
936     ax.legend()
937     ax.grid(True, alpha=0.3)
938
939     plt.tight_layout()
940     plt.savefig(f"{figure_dir}/{filename}", dpi=300, bbox_inches="tight")
941
942     plt.show()
943
944 #33: Plot Historical VaR at 95% confidence level
945
946 plot_var_model(
947     var_result=hist_95,
948     title="Historical 1-Day VaR at 95% Confidence Level",
949     filename="historical_var_95_threshold.png",
950     y_limits=(-0.16, 0.16)
951 )
952
953 #34: Plot GARCH(1,1)-Student-t VaR at 95% confidence level
954
955 plot_var_model(
956     var_result=garch_t_95,
957     title="GARCH(1,1)-Student-t 1-Day VaR at 95% Confidence Level",
958     filename="garch_t_var_95_threshold.png",
959     y_limits=(-0.16, 0.16)
960 )
961
962 #35: Plot Historical VaR at 99% confidence level
963
964 plot_var_model(
965     var_result=hist_99,
966     title="Historical 1-Day VaR at 99% Confidence Level",
967     filename="historical_var_99_threshold.png",
968     y_limits=(-0.16, 0.16)
969 )

```

```

970 #36: Plot GARCH(1,1)-Student-t VaR at 99% confidence level
971
972 plot_var_model(
973     var_result=garch_t_99,
974     title="GARCH(1,1)-Student-t 1-Day VaR at 99% Confidence Level",
975     filename="garch_t_var_99_threshold.png",
976     y_limits=(-0.27, 0.16)
977 )
978
979 #37: Observed vs expected exceedance rates for 95% VaR
980
981 bt_95 = backtesting_results[
982     backtesting_results["confidence_level"] == 0.95
983 ].copy()
984
985 bt_95 = bt_95.sort_values("coverage_error").reset_index(drop=True)
986
987 plt.figure(figsize=(12, 6))
988
989 plt.bar(
990     bt_95["model"],
991     bt_95["exceedance_rate"],
992     label="Observed exceedance rate"
993 )
994
995 plt.axhline(
996     0.05,
997     linestyle="--",
998     linewidth=1.5,
999     label="Expected exceedance rate"
1000 )
1001
1002 plt.title("Observed vs Expected Exceedance Rate for 95% VaR")
1003 plt.xlabel("Model")
1004 plt.ylabel("Exceedance Rate")
1005 plt.xticks(rotation=30, ha="right")
1006 plt.legend()
1007 plt.grid(True, axis="y", alpha=0.3)
1008 plt.tight_layout()
1009
1010 plt.savefig(f"{figure_dir}/exceedance_rate_95.png", dpi=300,
1011             bbox_inches="tight")
1012
1013 plt.show()
1014
1015 #38: Observed vs expected exceedance rates for 99% VaR
1016
1017 bt_99 = backtesting_results[
1018     backtesting_results["confidence_level"] == 0.99
1019 ].copy()
1020
1021 bt_99 = bt_99.sort_values("coverage_error").reset_index(drop=True)
1022
1023 plt.figure(figsize=(12, 6))
1024
1025 plt.bar(
1026     bt_99["model"],
1027     bt_99["exceedance_rate"],
1028     label="Observed exceedance rate"

```

```

1027 )
1028
1029 plt.axhline(
1030     0.01,
1031     linestyle="--",
1032     linewidth=1.5,
1033     label="Expected exceedance rate"
1034 )
1035
1036 plt.title("Observed vs Expected Exceedance Rate for 99% VaR")
1037 plt.xlabel("Model")
1038 plt.ylabel("Exceedance Rate")
1039 plt.xticks(rotation=30, ha="right")
1040 plt.legend()
1041 plt.grid(True, axis="y", alpha=0.3)
1042 plt.tight_layout()
1043
1044 plt.savefig(f"{figure_dir}/exceedance_rate_99.png", dpi=300,
1045             bbox_inches="tight")
1046
1047 #39: Rolling 252-day exceedances for Historical VaR and GARCH-t VaR
1048
1049 rolling_window = 252
1050 expected_exceedances_95 = rolling_window * 0.05
1051
1052 hist_95_rolling_exceedances = (
1053     hist_95["exceedance"]
1054     .astype(int)
1055     .rolling(rolling_window)
1056     .sum()
1057 )
1058
1059 garch_t_95_rolling_exceedances = (
1060     garch_t_95["exceedance"]
1061     .astype(int)
1062     .rolling(rolling_window)
1063     .sum()
1064 )
1065
1066 fig, ax = plt.subplots(figsize=(12, 5))
1067
1068 ax.plot(
1069     hist_95.index,
1070     hist_95_rolling_exceedances,
1071     linewidth=1.2,
1072     label="Historical VaR"
1073 )
1074
1075 ax.plot(
1076     garch_t_95.index,
1077     garch_t_95_rolling_exceedances,
1078     linewidth=1.2,
1079     label="GARCH(1,1)-Student-t VaR"
1080 )
1081
1082 ax.axhline(
1083     expected_exceedances_95,

```

```

1084     linestyle="--",
1085     linewidth=1.3,
1086     label=f"Expected exceedances = {expected_exceedances_95:.1f}"
1087 )
1088
1089 ax.set_title("Rolling 252-Day Number of Exceedances for 95% VaR")
1090 ax.set_xlabel("Date")
1091 ax.set_ylabel("Number of Exceedances")
1092
1093 ax.set_ylim(bottom=0)
1094 ax.margins(x=0)
1095
1096 ax.legend()
1097 ax.grid(True, alpha=0.3)
1098
1099 plt.tight_layout()
1100 plt.savefig(f"{figure_dir}/rolling_exceedances_95.png", dpi=300,
1101             bbox_inches="tight")
1102
1103 plt.show()
1104
1105 #40: Rolling 252-day exceedances for Historical VaR and GARCH-t VaR at
1106 99%
1107
1108 rolling_window = 252
1109 expected_exceedances_99 = rolling_window * 0.01
1110
1111 hist_99_rolling_exceedances = (
1112     hist_99["exceedance"]
1113     .astype(int)
1114     .rolling(rolling_window)
1115     .sum()
1116 )
1117
1118 garch_t_99_rolling_exceedances = (
1119     garch_t_99["exceedance"]
1120     .astype(int)
1121     .rolling(rolling_window)
1122     .sum()
1123 )
1124
1125 plt.figure(figsize=(14, 6))
1126
1127 plt.plot(
1128     hist_99.index,
1129     hist_99_rolling_exceedances,
1130     linewidth=1.2,
1131     label="Historical VaR"
1132 )
1133
1134 plt.plot(
1135     garch_t_99.index,
1136     garch_t_99_rolling_exceedances,
1137     linewidth=1.2,
1138     label="GARCH(1,1)-Student-t VaR"
1139 )
1140
1141 plt.axhline(
1142     expected_exceedances_99,

```

```

1140     linestyle="--",
1141     linewidth=1.5,
1142     label=f"Expected exceedances {expected_exceedances_99:.1f}"
1143 )
1144
1145 plt.title("Rolling 252-Day Number of Exceedances for 99% VaR")
1146 plt.xlabel("Date")
1147 plt.ylabel("Number of Exceedances")
1148 plt.legend()
1149 plt.grid(True, alpha=0.3)
1150 plt.tight_layout()
1151
1152 plt.savefig(f"{figure_dir}/rolling_exceedances_99.png", dpi=300,
1153             bbox_inches="tight")
1154 plt.show()

```